DEVSECOPS LAB BOOK • PHASE 9

OS & Infrastructure Hardening

Chapter 29 • Linux Security & CIS Benchmarks

6-7 hrs
Time Estimate

295+
CIS Controls

70%+
Attack Surface Reduction

Edition 2026 Updated • Ubuntu 22.04 LTS

Chapter Goal

Harden the operating system and infrastructure layer from the ground up: CIS Benchmarks, kernel hardening, SELinux/AppArmor, syscall filtering, and container escape prevention — reducing attack surface at the OS level.

BY THE END OF THIS CHAPTER

- ✓ CIS Benchmarks Level 1 & 2 applied (Ubuntu 22.04)
- ✓ Kernel hardening parameters optimized
- ✓ SELinux/AppArmor enforced with custom policies
- ✓ Syscall filtering (seccomp) blocking dangerous operations
- ✓ Container escape prevention mechanisms deployed
- ✓ Rootkit and malware detection active
- ✓ Compliance with security standards verified
- ✓ Audit logging capturing all system changes

6-7 hours

Time estimate

29.1 — Attack Surface Analysis

Kernel Exploits

- Memory corruption vulnerabilities
- Privilege escalation flaws
- Use-after-free bugs
- NULL pointer dereference

Weak Permissions

- World-writable directories
- SUID binaries with vulnerabilities
- Overly permissive ownership
- Unnecessary setgid files

Unneeded Services

- Listening on unnecessary ports
- Enabled but unused daemons
- Default credentials still set
- Network services without auth

Insecure Configs

- Swap encryption disabled
- Core dumps enabled (info leak)
- ICMP redirects allowed
- Unneeded IP forwarding

Missing Defenses

- No ASLR (predictable addresses)
- No seccomp (all syscalls allowed)
- No AppArmor/SELinux
- No audit logging

Hardening reduces: kernel exploit success rate by 70%+ • privilege escalation possibilities • attack persistence • lateral movement

Defense in Depth at the OS Level

1	**Filesystem Hardening**	Separate partitions, mount options, permissions
2	**Kernel Hardening**	Sysctl parameters, mitigations, disable unused features
3	**Service Hardening**	Disable unnecessary services, minimal installations
4	**Syscall Restrictions**	Seccomp, AppArmor, SELinux enforcing
5	**Monitoring & Detection**	Auditd, file integrity, rootkit scanning
6	**Incident Response**	Logging, forensics, recovery procedures

29.2 — What Are CIS Benchmarks?

Consensus-based security best practices developed by the Center for Internet Security — a community of experts and organizations, peer-reviewed and validated.

CIS Ubuntu 22.04 LTS Includes

- 295+ security recommendations
- Level 1: Basic hardening (achievable for all)
- Level 2: Advanced hardening (sensitive systems)
- Server profile & Workstation profile
- Automated and manual verification steps

Compliance Frameworks Using CIS

PCI-DSS	Requires CIS compliance
HIPAA	Recommends CIS
SOC 2	Leverages CIS controls
ISO 27001	Aligns with CIS
Gov. Standards	DISA STIGs based on CIS

CIS Benchmark Structure & Scoring

BENCHMARK STRUCTURE (7 SECTIONS)

- 1. Initial Setup — filesystem, /tmp /var/log, USB storage
- 2. Services — remove unnecessary services & protocols
- 3. Network Parameters — IPv6, TCP wrappers, firewall
- 4. Logging & Auditing — auditd, log permissions
- 5. Access Control — SUID/SGID, sudo, SSH config
- 6. System Maintenance — packages, permissions, users
- 7. Security Options — kernel params, AppArmor/SELinux

SCORING INTERPRETATION

Range	Rating
95-100%	Excellent (production-ready)
85-94%	Good (acceptable for most)
75-84%	Fair (needs improvement)
50-74%	Poor (significant gaps)
<50%	Critical (unacceptable risk)

REMEDIATION PRIORITIZATION

Severity	Timeline
Critical	Immediate (24 hrs)
High	Urgent (7 days)
Medium	Important (30 days)
Low	Nice-to-have (60+ days)

29.3 — Filesystem Hardening

Partition	Mount Options	Size
/ (root)	`defaults,noexec`	20 GB min
/boot	`nodev,nosuid,noexec`	1 GB
/tmp	`nodev,nosuid,noexec` ⚠️	5-10 GB
/var	`nodev,nosuid`	10-20 GB
/var/log	`nodev,nosuid,noexec`	10-50 GB
/var/tmp	`nodev,nosuid,noexec (bind → /tmp)`	= /tmp
/home	`nodev,nosuid`	varies

MOUNT OPTIONS EXPLAINED

`nodev`

Block special device files

`nosuid`

SUID/SGID bits ignored

`noexec`

No executable scripts/binaries

`ro`

Read-only (e.g. /boot post-boot)

`defaults`

Default mount options

⚠️ *Critical: noexec on /tmp prevents dropped-binary execution — a common exploitation technique.*

29.4 — Kernel Hardening Parameters

Memory & ASLR

`kernel.randomize_va_space`

2 (full ASLR)

`fs.suid_dumpable`

0 (no core dumps)

`kernel.core_uses_pid`

0

Visibility & BPF

`kernel.kptr_restrict`

2 (hide for all)

`kernel.dmesg_restrict`

1 (root only)

`kernel.unprivileged_bpf_disabled`

1

Process Control

`kernel.yama.ptrace_scope`

2 (admin only)

`kernel.sysrq`

0 (disabled)

`kernel.unprivileged_userns_clone`

0

Network Hardening

`net.ipv4.tcp_syncookies`

1 (SYN flood defense)

`net.ipv4.conf.all.accept_redirects`

0

`net.ipv4.conf.all.rp_filter`

1 (anti-spoof)

Also hardened: vm.swappiness=10 · vm.vfs_cache_pressure=50 · kernel.modules_disabled=1 · fs.protected_symlinks/hardlinks=1 · IP forwarding & source routing disabled · suspicious packets logged (log_martians)

29.5 — AppArmor Mandatory Access Control

AppArmor confines programs to a limited set of resources via per-application profiles — enforced (blocking) or complain (logging-only for testing).

SAMPLE PROFILE — database-server

```
capability setuid, setgid, net_bind_service,
capability sys_chroot,
network inet stream, inet dgram,

/var/lib/postgresql/** rwk,
/etc/postgresql/** r,

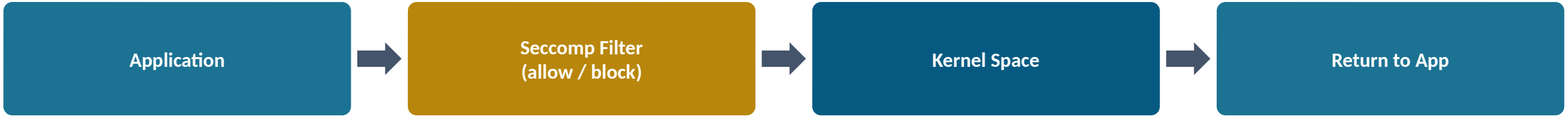
deny /boot/** rwx,
deny /sys/module/** rwx,
deny /proc/sys/** rwx,
```

PROFILE WORKFLOW

1. **`aa-complain profile`**
   Test in logging-only mode
2. **`aa-logprof`**
   Generate rules from observed behavior
3. **`apparmor_parser -r`**
   Load & reload the profile
4. **`aa-enforce profile`**
   Switch to enforcing (blocking)
5. **`aa-status / ausearch -m apparmor`**
   Verify & audit denials

29.6 — Seccomp (Syscall Filtering)

A syscall is the interface between user space and the kernel (open, read, write, fork, execve…). Seccomp filters which syscalls a process is allowed to make.



DANGEROUS SYSCALLS COMMONLY BLOCKED

- ptrace() — process tracing / escape
- execve() — execute new binary (in containers)
- init_module() / delete_module() — kernel modules
- reboot() — system reboot
- sysctl() — kernel parameter modification
- socket() — networking (unless needed)

SECCOMP PROFILE PATTERN

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "archMap": [{ "architecture":
     "SCMP_ARCH_X86_64" }],
  "syscalls": [
    { "names": ["read","write","open",
      "close","mmap","futex", ...],
      "action": "SCMP_ACT_ALLOW" }
  ]
}
```

29.7 — Container Escape Prevention

POD SECURITY STANDARDS — KEY CONTROLS

`runAsNonRoot: true`

Enforce non-root execution

`capabilities.drop: [ALL]`

Drop all Linux capabilities, add only what's needed

`seccompProfile: Localhost`

Apply custom restricted-syscalls filter

`readOnlyRootFilesystem`

Prevent writes to container root FS

`resources.limits`

cpu / memory caps prevent resource exhaustion

NETWORK POLICY — DEFAULT DENY MODEL

`default-deny-all`

Block all ingress & egress by default

`allow-from-ingress`

Permit traffic only from ingress-controller pods

`allow-to-database`

Egress restricted to postgresql on port 5432 + DNS

Zero-trust posture: only explicitly allowed traffic ever reaches or leaves a pod.

29.8-29.9 — Detection, Monitoring & Forensics

chkrootkit

Rootkit detection scan

AIDE

File integrity monitoring (baseline diff)

rkhunter

Rootkit Hunter — definitions-based scan

Lynis

Full security audit, grades A++ to F

ClamAV

Antivirus scanning & removal

AUDITD — WHAT'S MONITORED

- /etc/passwd, /etc/shadow, /etc/group — credential changes
- /etc/sudoers, /etc/sudoers.d/ — privilege changes
- /root/.ssh, /home — SSH key & home dir changes
- useradd/usermod/userdel — user administration
- execve syscalls (uid ≥ 1000) — command execution

KEY COMMANDS

```
ausearch -k passwd_changes
ausearch -ts recent
aureport --auth-summary
aureport --file-summary
-e 2   # make audit rules immutable
```

29.10-29.11 — Verification & Chapter Checklist

Filesystem

- ✓ Separate partitions
- ✓ nodev/nosuid/noexec
- ✓ Sticky bits set

Kernel

- ✓ Sysctl hardened
- ✓ ASLR = 2
- ✓ Core dumps disabled

Services & Access

- ✓ Unneeded services off
- ✓ SSH hardened
- ✓ Sudo configured

Security Mechanisms

- ✓ AppArmor enforced
- ✓ Seccomp deployed
- ✓ Firewall configured

Monitoring

- ✓ Auditd active
- ✓ AIDE baseline set
- ✓ Rootkit scans scheduled

HARDENING LEVEL & COMPLIANCE TARGETS

CIS L1	CIS L2	Container Escape Prevention	Rootkit Detection	Audit Log Retention
95%+ compliance	80%+ compliance	Implemented	Automated	6+ months

Key Takeaway — Defense in Depth

BEFORE HARDENING

- ✗ Default security settings
- ✗ All syscalls allowed
- ✗ Container can escalate to root
- ✗ No audit trail
- ✗ Rootkit installation possible

AFTER HARDENING

- ✓ CIS L1/L2 compliant
- ✓ Dangerous syscalls blocked
- ✓ Containers cannot escape
- ✓ Complete audit trail
- ✓ Rootkits detected immediately
- ✓ Attack surface reduced 70%+

Common Challenges & Solutions

Performance impact

Profile bottlenecks, tune swappiness, use tmpfs / SSD

App compatibility

Complain mode first, gradual enforcement, whitelisting

Scale across infra

IaC (Terraform/Ansible), baseline images, OpenSCAP scanning